

Nhúng phẳng

Gu Nan

Trường Trung học Nanshan, Miên Dương, Tứ Xuyên

Trại đông tin học 2007

Nguồn gốc: Nhúng phẳng

IOI China candidate team papers / 2007 / day 2 / Gu Nan

Abstract

Bài viết gồm hai phần chính. Phần thứ nhất giới thiệu thuật toán nhúng phẳng: các thao tác cụ thể, phân tích độ phức tạp, chứng minh tính đúng đắn và một số bài liên quan. Phần thứ hai là phụ lục, gồm tài liệu tham khảo và giả mã.

Từ khóa. Mặt phẳng; nhúng; hoạt động trong; liên quan; hoạt động ngoài; khối; cạnh gốc; cạnh hồi.

1 Thuật toán

1.1 Các định nghĩa liên quan

Trước hết nhắc lại vài khái niệm cơ bản.

1. **Vẽ phẳng:** một đồ thị có thể được biến đổi thành hình vẽ mà mọi cạnh không cắt nhau, trừ giao nhau tại đỉnh. Quá trình biến đổi ấy gọi là vẽ phẳng. Những đỉnh vốn kề nhau vẫn phải kề nhau sau khi biến đổi.
2. **Đồ thị phẳng:** đồ thị có thể vẽ phẳng được gọi là đồ thị phẳng; ngược lại là đồ thị không phẳng.
3. **Nhúng phẳng:** tương đương với vẽ phẳng, nhưng được mô tả dưới dạng lưu trữ: với mỗi đỉnh, ta lưu danh sách các đỉnh kề theo thứ tự chiều kim đồng hồ. Bảng dùng để lưu gọi là danh sách kề.

Trong bài này, mọi lần nói đến “mặt phẳng” đều hiểu theo các định nghĩa trên, không phải mặt phẳng hình học tùy ý.

Hình 1: ví dụ một đồ thị được vẽ lại sao cho các cạnh không cắt nhau.

1.2 Mục tiêu của thuật toán

Với một đồ thị G có n đỉnh và m cạnh, mục tiêu là dùng thời gian $\mathcal{O}(n)$ để phán đoán đồ thị có phẳng hay không. Nếu đồ thị phẳng, thuật toán cũng phải dựng được một nhúng phẳng trong thời gian $\mathcal{O}(n)$.

1.3 Kiến thức chuẩn bị

Ta cần các kiến thức như tìm kiếm theo chiều sâu, thành phần song liên thông, đỉnh khớp và sắp xếp đếm. Ngoài ra cần một vài định lý cơ bản: số cạnh của một đồ thị phẳng không vượt quá $3n - 6$; nếu vượt ngưỡng đó thì đồ thị chắc chắn không phẳng. Kuratowski chứng minh rằng hai đồ thị cản trở nhúng phẳng là K_5 và $K_{3,3}$. Một đồ thị chứa một đồ thị đồng phôi với K_5 hoặc $K_{3,3}$ thì không phẳng, và chiều ngược lại cũng đúng.

Thao tác cơ bản của thuật toán là thêm cạnh. Ta tạo một đồ thị G_P , rồi đưa các cạnh của đồ thị gốc vào G_P theo một thứ tự nhất định. Thao tác đó được gọi là **nhúng cạnh**.

Hình 2: hai chuồng ngựa cổ điển cho tính phẳng, K_5 và $K_{3,3}$.

1.4 Tổng quan thuật toán

Để trình bày rõ ràng, trước hết chỉ xét nhúng phẳng trên đồ thị song liên thông. Với đồ thị có đỉnh khớp, ta có thể tách đồ thị tại các đỉnh khớp, nhúng từng phần rồi ghép lại.

Trước tiên thực hiện một lần DFS, thu được cây DFS và thứ tự DFS đảo ngược DFI. Cạnh nối một đỉnh với con của nó gọi là **cạnh cây**; cạnh nối một đỉnh với hậu duệ không phải con trực tiếp gọi là **cạnh hồi**.

Sau đó xử lý các đỉnh theo thứ tự DFI đảo ngược. Khi xử lý một đỉnh, ta trước hết nhúng các cạnh cây của nó vào G_P , rồi nhúng mọi cạnh hồi của nó. Các cạnh nối với tổ tiên chưa cần xử lý. Nếu quá trình nhúng thất bại, ta kết luận G không phẳng; nếu không, ta hoàn thành nhúng phẳng.

1.5 Nhúng cạnh

Nhúng cạnh là thao tác quan trọng nhất. Giả sử cần nhúng cạnh (v, w) . Ban đầu, thành phần liên thông đang xét chứa một đỉnh khớp r . Nếu bỏ r , thành phần này tách thành hai phần. Ta gán một bản sao của r cho mỗi phần, nhúng cạnh (v, w) , rồi hợp nhất hai thành phần lại; lúc đó r không còn là đỉnh khớp trong khối mới.

Hình 3: tách một khối tại đỉnh khớp, nhúng cạnh mới, có thể đảo một phía rồi hợp nhất lại.

Trong quá trình nhúng, hễ G_P chứa đỉnh khớp thì ta tách thành phần liên thông của nó thành hai phần. Khi nhúng cạnh hồi, các phần này lại được ghép. Vì vậy G_P thực chất được tạo bởi nhiều thành phần song liên thông. Khi cài đặt, đây không chỉ là cách hiểu mà là cấu trúc cần được xử lý thật sự.

1.6 Hoạt động ngoài, liên quan và hoạt động trong

Phép đảo được dùng để bảo vệ một tính chất quan trọng: trong quá trình nhúng cạnh, mọi đỉnh hoạt động ngoài phải nằm trên mặt ngoài.

Mặt ngoài là mặt tiếp xúc với miền vô hạn bên ngoài đồ thị. Với một thành phần song liên thông chỉ có một cạnh, ta xem nó như một chu trình suy biến để thao tác trên mặt ngoài vẫn thống nhất.

Hình 4: cách xem một cạnh đơn như một chu trình ngoài suy biến.

Định nghĩa đỉnh hoạt động ngoài phức tạp hơn. Trong một thành phần song liên thông, đỉnh có thứ tự DFI nhỏ nhất được gọi là **đỉnh gốc** của thành phần. Do phép tách chỉ sinh ra khi nhúng cạnh cây, khi một đỉnh khớp bị tách thành nhiều bản sao, bản sao nằm cùng phần với một con sẽ trở thành gốc của phần đó. Những đỉnh này gọi là **điểm sao chép**; bản duy nhất không phải gốc gọi là **điểm gốc ban đầu**.

Giả sử r là gốc của một thành phần song liên thông B . Khi đó trong B , r chỉ có một con. Nếu r có hai con c_1, c_2 , do thứ tự DFS, khi duyệt từ c_1 ta có thể đi trong B tới c_2 mà không qua r , mâu thuẫn với việc c_2 là con của r . Gọi con duy nhất ấy là c ; cạnh rc gọi là **cạnh gốc**, còn thành phần tương ứng gọi là **khối**. Khối gắn với bản sao gốc được gọi là **khối con** của điểm gốc ban đầu.

Khi đang xử lý đỉnh v , một đỉnh đã xử lý được gọi là **hoạt động ngoài** nếu thỏa ít nhất một trong hai điều kiện:

1. Nó có cạnh trực tiếp tới một tổ tiên của v .
2. Trong một khối con của nó tồn tại đỉnh hoạt động ngoài.

Hình 5: các đỉnh hình vuông là hoạt động ngoài; tính hoạt động được truyền ngược qua các khối con.

Để lấy nhanh thông tin này, với mỗi đỉnh ta lưu:

$ecpoint(x)$ = độ sâu tổ tiên sớm nhất mà x nối trực tiếp tới,

$lowpoint(x)$ = độ sâu tổ tiên sớm nhất mà x hoặc hậu duệ của x nối tới.

Mỗi đỉnh còn có một danh sách hai chiều $SDlist$, lưu các giá trị $lowpoint$ của các con theo thứ tự tăng dần. Vì giá trị này bị chặn bởi n , có thể dùng sắp xếp đếm để xây dựng mọi danh sách trong thời gian tuyến tính. Khi cần biết một đỉnh có hoạt động ngoài hay không, chỉ cần kiểm tra $ecpoint$ và phần tử đầu của $SDlist$ có nhỏ hơn độ sâu của đỉnh đang xử lý hay không. Khi một con được hợp nhất vào cha, ta xóa giá trị tương ứng khỏi $SDlist$ của cha.

Tương tự, một đỉnh **liên quan** khi đang xử lý v nếu:

1. Nó có cạnh trực tiếp tới v .
2. Trong một khối con của nó tồn tại đỉnh liên quan.

Một đỉnh có thể vừa hoạt động ngoài vừa liên quan. Đỉnh **hoạt động trong** là đỉnh liên quan nhưng không hoạt động ngoài; đỉnh **không hoạt động** là đỉnh không thuộc hai loại trên. Tương ứng có khối hoạt động ngoài, khối liên quan, khối hoạt động trong và khối không hoạt động. Hàm walkup sẽ thu thập các thông tin này.

1.7 Phép đảo

Mục đích của phép đảo là giữ mọi đỉnh hoạt động ngoài trên mặt ngoài trong suốt quá trình nhúng. Trong ví dụ ở hình 3, vì một đỉnh như y là hoạt động ngoài, phần dưới phải được đảo trước khi nhúng cạnh.

Cách trực tiếp là đảo toàn bộ danh sách kề của mọi đỉnh trong khối, nhưng như vậy quá chậm. Ta dùng cạnh gốc: mọi cạnh cây ban đầu mang giá trị $+1$. Khi một khối cần đảo, ta đổi giá trị cạnh gốc của nó thành -1 . Một đỉnh có bị đảo hay không phụ thuộc vào số cạnh -1 trên đường cây từ gốc tới nó; số đó lẻ thì cần đảo, chẵn thì không. Sau khi xử lý xong mọi đỉnh, một DFS cuối cùng sẽ khôi phục toàn bộ thông tin đảo.

Vì các đỉnh được xử lý theo thứ tự DFI đảo ngược và mọi đỉnh hoạt động ngoài nằm trên mặt ngoài, mỗi lần thêm cạnh hồi đều diễn ra trên mặt ngoài. Do đó khi xử lý một đỉnh, ta chỉ cần duyệt mặt ngoài của các khối. Mỗi đỉnh trên mặt ngoài lưu hai đỉnh kề ngoài của nó. Khi một khối bị đảo, ta chỉ cần hoán đổi thông tin kề ngoài thích hợp; không cần biết tuyệt đối đang đi theo chiều kim đồng hồ hay ngược chiều, chỉ cần biết đi vào từ phía nào thì đi ra phía còn lại.

Hình 6: phép đảo đổi hướng một khối để đỉnh hoạt động ngoài vẫn nằm trên mặt ngoài sau khi ghép.

1.8 Phân tích toàn bộ quá trình

Khởi tạo G_P là đồ thị rỗng. Sau đó xử lý từng đỉnh theo thứ tự DFI đảo ngược. Khi xử lý v , với mỗi cạnh hồi (v, w) , ta đi từ w dọc mặt ngoài lên tới v để ghi lại thông tin liên quan; thao tác này là walkup. Sau đó đi từ v xuống để nhúng cạnh; thao tác này là walkdown. Nếu cuối cùng có cạnh hồi chưa được nhúng, đồ thị không phẳng; ngược lại thu được những phẳng.

Khi cần nhúng cạnh (v, w) , do danh sách kề lưu theo chiều kim đồng hồ, nếu đang duyệt theo chiều kim đồng hồ thì thêm w vào đầu danh sách kề của v ; nếu ngược chiều thì thêm vào cuối. Các cạnh cây ban đầu được thêm thống nhất vào đầu danh sách.

DFS(G), tính lowpoint cho mọi đỉnh

Khởi tạo G_P , SDlist và sắp xếp các SDlist

for v theo thứ tự DFI đảo ngược:

for mọi con c của v :

nhúng cạnh gốc (v, c) vào G_P

for mọi cạnh hồi (v, w) :

Walkup(G_P , v , w)

for mọi con c của v :

Walkdown(G_P , c)

for mọi cạnh hồi (v, w) :

if (v, w) chưa nằm trong G_P :

return (không phẳng, G_P)

RecoverPlanarEmbedding(G_P)

return (phẳng, G_P)

1.9 walkup

walkup dùng để thu thập thông tin khi xử lý đỉnh v . Với cạnh hồi (v, w) , ta đi từ w dọc mặt ngoài lên tới v . Mỗi đỉnh có một danh sách *proots* ghi các khối con liên quan. Khi đi từ một gốc bản sao lên điểm gốc ban đầu r , ta thêm bản sao đó vào *proots*(r); khi đó r trở thành đỉnh liên quan.

Từ w có thể đi theo hai hướng trên mặt ngoài. Hai đường này có thể chênh lệch rất lớn, nên thuật toán duyệt đồng thời hai hướng; nhiều nhất sau hai lần độ dài đường ngắn hơn sẽ tới gốc khối. Ngoài ra, các cạnh hồi khác nhau có thể tạo ra phần đường trùng nhau. Vì thế mỗi đỉnh có một nhãn *visited*. Khi xử lý v , mọi đỉnh đã thăm được gán *visited* = v ; nếu gặp lại đỉnh như vậy thì dừng phần duyệt hiện tại.

Hình 7: walkup đi đồng thời theo hai hướng và dừng khi gặp gốc khối hoặc phần đã thăm.

1.10 walkdown

walkdown dùng thông tin liên quan để nhúng các cạnh hồi. Nó cũng thao tác trên mặt ngoài, nhưng khác với walkup: khi duyệt xuống, thông thường không được đi qua đỉnh hoạt động ngoài. Ngoại lệ duy nhất là khi đỉnh đó vẫn còn là đỉnh liên quan và ta cần đi xuống khối con của nó.

Khi xử lý v , ta đi xuống từ từng khối con theo hai hướng: thuận và ngược. Nếu gặp một đỉnh có *proots* không rỗng thì đi xuống khối con liên quan của nó, nhưng phải chọn lại hướng.

Nếu trong quá trình đi xuống gặp một đỉnh hoạt động ngoài nhưng không liên quan, ta dừng; đỉnh đó gọi là **đỉnh dừng**. Để tránh dừng quá sớm, thứ tự duyệt phải tuân theo hai quy tắc:

1. Khi có nhiều khối con cần đi xuống, luôn đi xuống khối hoạt động trong trước; sau khi quay lại

mới xử lý các khối còn lại. Vì khối hoạt động trong có $lowpoint = v$, còn khối hoạt động ngoài có $lowpoint < v$, chỉ cần đưa các khối có $lowpoint = v$ vào đầu danh sách *proots*.

2. Khi đi xuống một khối liên quan và hoạt động ngoài, trước hết chọn hướng trực tiếp tới đỉnh hoạt động trong; nếu không có thì chọn hướng trực tiếp tới đỉnh vừa liên quan vừa hoạt động ngoài. Khi gặp đỉnh dừng hoặc gốc khối thì dừng.

Khi đi tới một đỉnh w , các trường hợp được xét theo thứ tự:

1. Nếu w là đỉnh liên quan, trước hết nhúng cạnh (v, w) .
2. Nếu w không hoạt động ngoài, đi tiếp tới đỉnh kế tiếp.
3. Nếu w hoạt động ngoài nhưng không liên quan, dừng lần duyệt này.
4. Nếu w vừa liên quan vừa là đỉnh khớp, đi xuống khối con của nó.

Hình 8: sau khi nhúng cạnh tới w , nếu w vẫn còn liên quan thì tiếp tục đi xuống khối con; nếu không, w trở thành đỉnh dừng.

Để hợp nhất các thành phần song liên thông sau khi nhúng cạnh, khi đi xuống một khối con ta đẩy lên ngăn xếp bốn thông tin: điểm không sao chép, điểm sao chép, hướng đi xuống và hướng quay lại. Khi hợp nhất, nếu hai hướng mâu thuẫn thì cần đảo khối trước khi ghép.

Còn một tối ưu hóa quan trọng. Nếu gặp đỉnh dừng s , đoạn mặt ngoài từ một đỉnh hoạt động trong p tới s thường không còn ích lợi nhưng vẫn có thể bị duyệt lại. Ta thêm một cạnh giả (v, s) để bỏ qua đoạn này trong các lần duyệt tiếp theo. Mỗi khối song liên thông nhiều nhất gặp hai đỉnh dừng, nên tổng số cạnh giả không vượt quá $2n$; cuối thuật toán chỉ cần xóa chúng.

Hình 9: cạnh giả nối tới đỉnh dừng giúp tránh duyệt lại một đoạn mặt ngoài không còn cần thiết.

2 Độ phức tạp

DFS và tính $lowpoint$ hiển nhiên là tuyến tính. Việc xây dựng mọi *SDlist* cũng tốn $\mathcal{O}(n)$, và mỗi lần xóa phần tử khỏi danh sách này là hằng số nên tổng vẫn là $\mathcal{O}(n)$. Phép đảo chỉ xảy ra khi hợp nhất thành phần song liên thông, nhiều nhất n lần, mỗi lần hằng số. Nhúng mỗi cạnh cũng là hằng số.

Trong walkup và walkdown, các đoạn mặt ngoài đã đi qua sẽ bị cập nhật bởi cạnh được nhúng và không bị duyệt lại. Khi duyệt khối hoạt động trong tuy không thêm cạnh giả, khối ấy cũng không bị duyệt lần thứ hai vì nó không có cạnh nối với tổ tiên sớm hơn. Do đó mỗi cạnh chỉ bị duyệt số lần hằng số. Vì đồ thị phẳng có không quá $3n - 5$ cạnh, cộng thêm $2n$ cạnh giả vẫn là tuyến tính. Tổng độ phức tạp của toàn bộ thuật toán là $\mathcal{O}(n)$.

3 Chứng minh đúng đắn

Nếu thuật toán sai thì mọi cải tiến về độ phức tạp đều vô nghĩa, nên phần này rất quan trọng.

Ngoài ngoại lệ đã nêu, walkdown không đi qua đỉnh hoạt động ngoài; vì vậy khi nhúng cạnh nó không che mất các đỉnh hoạt động ngoài. Nếu mọi đỉnh hoạt động ngoài luôn nằm trên mặt ngoài, các cạnh mới không cắt nhau.

Với đồ thị không phẳng, ta chỉ cần chỉ ra trong mỗi tình huống thất bại đều có một đồ thị con đồng phôi với K_5 hoặc $K_{3,3}$.

Tình huống 1. Một đỉnh có ít nhất ba khối con vừa liên quan vừa hoạt động ngoài. Do chỉ duyệt hai hướng trên mặt ngoài, nhiều nhất hai khối như vậy có thể được xử lý trước khi dừng. Khi xuất hiện ba khối, ta tìm được một $K_{3,3}$ đồng phôi: ba đỉnh phía trên là (u, v, r) , ba nhánh phía dưới đến từ ba khối con (a, b, c) .

Hình 10: ba khối con hoạt động ngoài tạo thành một $K_{3,3}$ đồng phôi.

Tình huống 2. Khi một đỉnh hoạt động ngoài w vẫn là đỉnh liên quan, thuật toán có thể đi qua w xuống khối con. Việc này không làm mất w khỏi mặt ngoài: mọi cạnh được nhúng ở một phía của khối con, và sau khi hợp nhất w không còn giữ khối con đó nữa.

Hình 11: đi qua một đỉnh hoạt động ngoài để xử lý khối con của nó không che mất đỉnh đó khỏi mặt ngoài.

Nếu một khối con của w cần được duyệt hai lần thì đồ thị không phẳng. Có ba trường hợp con:

1. Trong khối con hoặc hậu duệ của nó có một đỉnh dừng tách hai đỉnh liên quan trở lên. Khi đó tồn tại một $K_{3,3}$ đồng phôi, với hai phía tương ứng như (u, a, c) và (v, w, b) .
2. Trong khối con hoặc hậu duệ của w có hai đỉnh vừa liên quan vừa hoạt động ngoài. Khi đó tìm được một K_5 đồng phôi.
3. w có hai khối con vừa liên quan vừa hoạt động ngoài. Nếu mỗi khối chỉ cần một lần duyệt thì w vẫn nằm trên mặt ngoài; nếu có ba khối thì quay về tình huống 1; nếu một khối cần hai lần duyệt thì quay về hai trường hợp con trên.

Hình 12 và 13: các cấu hình buộc một khối con phải duyệt hai lần lần lượt sinh ra $K_{3,3}$ hoặc K_5 đồng phôi.

Tình huống 3. Một đỉnh liên quan không thể được duyệt tới vì bị hai đỉnh dừng chặn lại. Khi đó tồn tại một $K_{3,3}$ đồng phôi, chẳng hạn bởi hai bộ nhánh (u, r, c) và (v, a, b) .

Hình 14: một đỉnh liên quan bị hai đỉnh dừng chặn dẫn tới $K_{3,3}$ đồng phôi.

Như vậy mọi nguyên nhân làm quá trình nhúng thất bại đều chứng minh được đồ thị không phẳng; còn nếu không thất bại, các cạnh được thêm trên mặt ngoài và không cắt nhau. Thuật toán đúng.

4 Bài liên quan và tổng kết

Một số bài liên quan:

1. Trại đông 2004: *Bấp ngô tổ ong*.
2. MIPT 090: *Planarity graph*.
3. Bài kiểm thử đi kèm: *Bảng mạch*, kèm dữ liệu và mã nguồn.

Thuật toán này có thể không xuất hiện thường xuyên trong thi đấu, nhưng việc học nó giúp hiểu sâu hơn về đồ thị phẳng. Đối mặt với những bài khó hơn cũng là cách rèn năng lực phân tích và cài đặt.

5 Phụ lục

Tài liệu tham khảo chính:

John M. Boyer, *Simplified $O(n)$ Planarity by Edge Addition*.

Trong giả mã, ký hiệu $\leftarrow$ là phép gán. Với đỉnh k , k_{in} là hướng đi vào, k_{out} là hướng đi ra. Trên mặt ngoài, hướng được mã hóa bằng 0 và 1. Nếu một đỉnh có điểm sao chép, và khối của điểm sao chép ấy có con duy nhất c , ta lưu điểm sao chép dưới dạng $c + n$.

Các hàm phụ.

GetSuccessorOnExternalFace(v , v_{in}) trả về đỉnh kế tiếp trên mặt ngoài.

GetActiveSuccessorOnExternalFace(v , v_{in}) trả về đỉnh hoạt động kế tiếp trên mặt ngoài.

MergeBiconnectedComponent(S) hợp nhất các khối được lưu trên ngăn xếp, gồm cả xử lý phép đảo.

EmbedBackEdge(v , v_{in} , w , w_{in}) nhúng một cạnh hồi.

Walkup(w , v) đi từ w lên v .

Walkdown(v) đi từ v xuống để nhúng các cạnh hồi.

MergeBiconnectedComponent

Lay tu stack S bốn giá trị: r , r_{in} , w , w_{out}

if $r_{\text{in}} == w_{\text{out}}$:

 hoan doi hai hướng của w

 đặt cạnh góc (r , w) thành -1

$w_{\text{out}} \leftarrow w_{\text{out}} \text{ xor } 1$

Xóa w khỏi proots của r

Xóa ($w - n$) khỏi SDlist của cha nó

$x \leftarrow 1 \text{ xor } w_{\text{out}}$

Đặt hướng v_{in} của v trở lại x

Đặt hướng của x đang trở lại w thành trở lại v

walkup

Đang xử lý đỉnh V , cần đi từ w lên V .

Danh dấu cơ cạnh hồi của w bằng V

$(x, x_{\text{in}}) \leftarrow (w, 1)$

$(y, y_{\text{in}}) \leftarrow (w, 0)$

while $x \neq V$:

 if $\text{visited}[x] == V$ or $\text{visited}[y] == V$:

 break

$\text{visited}[x] \leftarrow V$

$\text{visited}[y] \leftarrow V$

 if x là góc:

```

        z1 <- x
    else if y là góc:
        z1 <- y
    else:
        z1 <- nil
    if z1 != nil:
        c <- z1 - n
        if lowpoint[c] < depth[V]:
            them z1 vào cuối proots[z]
        else:
            them z1 vào đầu proots[z]
    else:
        (x, x_in) <- GetSuccessorOnExternalFace(x, x_in)
        (y, y_in) <- GetSuccessorOnExternalFace(y, y_in)

```

walkdown

Đang xử lý đỉnh v , đỉnh gốc được ký hiệu v_0 ; cần đi xuống từ v_0 .

Xóa rỗng stack S

```

for v_out in {0, 1}:
    (w, w_in) <- GetSuccessorOnExternalFace(v)
    while w != v0:
        if backedge_flag[w] == v:
            while S không rỗng:
                MergeBiconnectedComponent(S)
            EmbedBackEdge(v0, v_out, w, w_in)
            xóa backedge_flag[w]

    if proots[w] không rỗng:
        day (w, w_in) vào S
        w0 <- phần tử đầu của proots[w]
        (x, x_in) <- GetActiveSuccessorOnExternalFace(w0, 0)
        (y, y_in) <- GetActiveSuccessorOnExternalFace(w0, 1)

        if x là đỉnh hoạt động trong:
            (w, w_in) <- (x, x_in)
        else if y là đỉnh hoạt động trong:
            (w, w_in) <- (y, y_in)
        else if x là đỉnh liên quan:
            (w, w_in) <- (x, x_in)
        else:
            (w, w_in) <- (y, y_in)

    if w == x:
        w_out <- 0
    else:
        w_out <- 1

```



```

        day (w0, w_out) vao S

    else if w la dinh hoạt động trong:
        (w, w_in) <- GetSuccessorOnExternalFace(w, w_in)
    else:
        if lowpoint[c] < depth[v] and S rong:
            nhưng canh gia (v0, w)
            break

if S không rong:
    break

```